

CSA1715-ARTIFICIAL INTELLIGENCE

LAB PROGRAM- 14

06-08-2026

14. Write the python program to implement Alpha & Beta pruning algorithm for gaming.

Aim

To implement the Alpha-Beta Pruning Algorithm in Python for game playing. The objective is to improve the efficiency of the Minimax algorithm by eliminating branches of the game tree that cannot influence the final decision, thereby reducing the number of nodes evaluated.

Algorithm

1. Start the program.
2. Define the terminal values of the game tree.
3. Initialize Alpha to negative infinity.
4. Initialize Beta to positive infinity.
5. Evaluate the MAX node and update Alpha.
6. Evaluate the MIN node and update Beta.
7. Compare Alpha and Beta during the evaluation.
8. If Alpha is greater than or equal to Beta, stop evaluating the remaining branch.
9. Mark the unnecessary branch as pruned.
10. Determine the optimal value.
11. Display the optimal value and pruning information.
12. Generate and display the game tree using NetworkX and Matplotlib.
13. Stop the program.

Program

```
import networkx as nx  
import matplotlib.pyplot as plt
```

```
v = [3,5,2,9]
```

```
alpha = float('-inf')
```

```
left = min(v[0],v[1])
```

```
alpha = max(alpha,left)
```

```
right = v[2]
```

```
pruned = alpha >= right
```

```
result = max(left,right)
```

```
print("Optimal Value:",result)
```

```
print("9 branch pruned:",pruned)
```

```
G = nx.DiGraph()
```

```
G.add_edges_from([  
    ("MAX","MIN1"),("MAX","MIN2"),  
    ("MIN1","3"),("MIN1","5"),  
    ("MIN2","2")  
])
```

```
p = {"MAX":(0,2),"MIN1":(-1,1),  
     "MIN2":(1,1),"3":(-1.5,0),  
     "5":(-.5,0),"2":(.5,0)}
```

```
nx.draw(G,p,with_labels=True,node_size=1800,  
        node_color="lightgreen")
```

```
plt.text(1.5,0,"9\nPRUNED",ha="center",fontsize=12)
```

```
plt.title("Alpha-Beta Pruning")
```

```
plt.show()
```

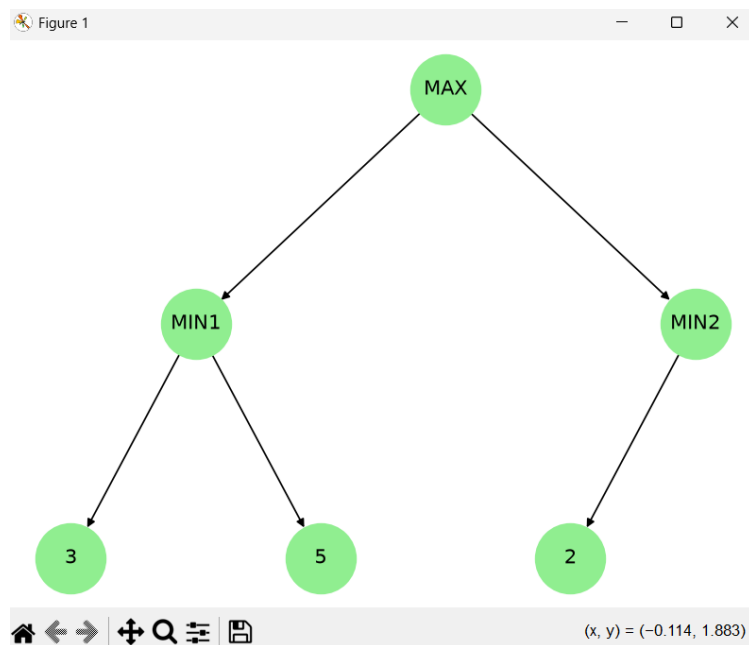
Input:

Leaf values: 3 5 2 9

Output:

Optimal Value: 3

9 branch pruned: True

**Result:**

The Alpha-Beta Pruning Algorithm was implemented successfully in Python. The algorithm efficiently determined the optimal game value by eliminating unnecessary branches during the search process, thereby reducing computation while producing the same result as the Minimax algorithm.